

A01712379

Laura Cintora Cendejas

Reflexión Individual

Tecnológico de Monterrey Campus Querétaro

Estructuras de datos jerárquicas

Actividad Integradora 3

Para poder desarrollar alguna aplicación, se requiere un procesamiento que sea eficiente para las grandes cantidades de datos, por lo tanto, tomar la decisión de una estructura indicada va a ver la diferencia significativa en el rendimiento debido a la complejidad del código. Bueno, en esta actividad que realice junto con mi compañero Farid, consistió en que ordenáramos, analizáramos y extraer información de un archivo, específicamente está enfocada en los accesos por dirección IP.

Por ello se implementó en el código un Binary Heap, como una estructura jerárquica principal, por ello en este documento se explicará sobre su elección, ventajas y su eficiencia a comparación de otras estructuras.

Las estructuras de datos jerárquicas, ya sea los árboles u otros, nos ayuda a organizar la información de manera que tal se puedan realizar las operaciones como inserción, eliminación o búsqueda de una manera más eficiente. Por lo tanto, el heap binario mantiene en todos los momentos el elemento más relevante, por ejemplo la IP, con un alto número de accesos en la raíz, por lo que minimiza de forma considerable el tiempo de acceso a los datos más significativos.

En esta actividad integradora se requería en primer lugar ordenar los registros por IP, luego contabilizar accesos por IP, después identificar las 10 IPs con mayor accesos y por último encontrar la IP con menos accesos, es decir mayor o igual que 3. Por lo que Binary Heap realiza las complejidades para resolver esta actividad integradora, ya que prioriza el orden parcial de los datos y facilita su actualización continua.

Un Binary Search Heap es una de las estructuras que nos permitió realizar los tres elementos esenciales en tiempo promedio de  $O(\log n)$ , ya que si el árbol se encuentra balanceado, pero si no se encuentra se llega a degenerar con un  $O(n)$ . Sin embargo, el Binary Heap está diseñado para soportar la extracción de los elementos relevantes y significativos, ya sea mínimo o máximo, y esto no lo garantiza la complejidad de  $O(\log n)$ .

Cabe mencionar que el Binary Heap es el adecuado para poder recuperar de una forma veloz los valores extremos de máximo o mínimo y mantener la actualización de forma constante. Ahora inserción (insert) esta es una complejidad computacional de las operaciones por lo que cada vez se inserta un nuevo registro o elemento en el Binary Heap, ya que se agrega al final de un arreglo para después reorganizarlo hasta la parte superior y mantenerlo dentro de la propiedad de heap por lo que su complejidad es  $O(\log n)$ .

Ahora también se usó uno que elimina la raíz, es decir, el elemento con alta prioridad se reemplaza con el último para después reorganizarlo hacia la parte inferior de heapify down por lo que su complejidad de esta extracción del elemento prioritario es igual que la de insertar.

Ahora otra de ellas son la recuperación del máximo, mínimo en este caso están como peek max y peek min, por lo que estas operaciones solo acceden a la raíz o al último elemento del árbol todo dependiendo del tipo del heap ya si tienen un  $O(1)$  puede acceder a la raíz de forma directa, pero si tienen un  $O(n)$  se buscará un mínimo o máximo no ordenado.

Se realizó también una reordenación completa el cual se aplicó un algoritmo Heap Sort sobre los datos, asimismo se tiene una complejidad de  $O(n \log n)$  este es el ordenamiento. Ya para finalizar se hizo y se implementó la búsqueda condicional que está en el código como find\_min\_if en donde buscamos el mínimo, ya que se espera que cumpla con una condición, entonces como heap no es un BST en este caso tienen que recorrer todas las hojas que existan por lo que su complejidad es  $O(n)$  el cual este conjunto de operaciones se demuestra el uso Binary Heap de este problema resulto ser eficiente con base al tiempo y además con la prioridad de operación que requería para llevarlo de forma correcta.

A continuación se explica y se muestra el pseudocódigo para buscar la IP con un menor número de acceso mayor o igual a 3:

const T& Heap<T>::find\_min\_if(F fun) const → La función busca el elemento mínimo del heap y que además cumpla con una condición específica.

if (empty()) throw std::out\_of\_range("Heap vacío"); → Aquí se realiza la verificación de que si el heap se encuentra vacío, de no ser así se envía un mensaje, ya que se busca una estructura vacía.

const T\* min\_ptr = nullptr; → Bueno es esta línea se está creando un puntero que se va a almacenar la dirección de este elemento por lo que cumpla la condición y además es el menor entre ellos, además uso de nullptr para indicar que aún no se encuentra ninguno.

for (size\_t i = size / 2; i < size; ++i) → Aquí es la búsqueda entre las hojas del heap esto quiere decir que están en el índice del tamaño entre dos hasta el tamaño de menos uno, entonces en otras palabras empieza desde este punto el recorrido de las hojas.

```
if (fun(data[i])) {  
    if (!min_ptr || data[i] < *min_ptr) {  
        min_ptr = &data[i]; → Se realiza la comparación y condición de este elemento si  
es que llega a cumplir la posición de la condición que se quiere llegar.
```

if (!min\_ptr) → Aquí si no se encuentra nada entre las hojas, es decir, un candidato válido comienza a revisar los nodos internos.

```
{  
    for (size_t i = 0; i < size / 2; ++i) → Se hace una búsqueda entre los nodos padre del  
heap por lo que se va recorriendo desde el índice 0 hasta llegar el size / 2 - 1 esto quiere  
decir la parte alta o mayor del heap.
```

```
if (fun(data[i])) { → aquí se aplica la misma lógica de antes si cumple con la
condición y en este caso es menor que el actual, entonces se actualiza min_ptr.
    if (!min_ptr || data[i] < *min_ptr) {
        min_ptr = &data[i];
```

```
if (!min_ptr)
    throw std::invalid_argument("Ningún elemento cumple la condición"); → Envía
mensaje si no cumple con la condición
```

Busca el elemento más pequeño dentro del heap que pueda cumplir una condición específica, por lo tanto, primero verifica y revisa las hojas y además, si no existe ninguno en los nodos internos, ya lanza un mensaje.

En conclusión, esta actividad desarrollo duro más de lo que uno tenía pensado, es decir, ha sido una de las que más tiempo se lleva uno realizando opero, aquí nos damos la cuenta de la importancia de elegir una estructura adecuada para los datos de acuerdo con el problema. Por lo que Binary Heaop fue la opción correcta para gestionar nuestras prioridades en donde se requiere de forma rápida acceder a un elemento importante a comparación un BTS, ya que tomamos en cuenta los tiempos de máximos de inserción y la extracción, sobre todo para este caso de los accesos de IP.

Por lo tanto, el algoritmo de ordenamiento que implementamos y la búsqueda condicional logro que nuestro código se ejecutara de forma eficaz. Siempre que avanzo en esta materia ha sido un reto, pero cada vez aprendo más y más.

#### Referencias:

Improve, K. F. (2013, marzo 16). Heap sort - data structures and algorithms tutorials. GeeksforGeeks. <https://www.geeksforgeeks.org/heap-sort/>

Heap data structure. (2024, enero 24). GeeksforGeeks. <https://www.geeksforgeeks.org/heap-data-structure/>

Date and time parsing in C++. (2023, septiembre 22). GeeksforGeeks. <https://www.geeksforgeeks.org/date-and-time-parsing-in-cpp/>

Improve, K. F. (2013a, marzo 7). Insertion sort algorithm. GeeksforGeeks. <https://www.geeksforgeeks.org/insertion-sort-algorithm/>

Improve, K. F. (2014, enero 28). Binary search algorithm - iterative and recursive implementation. GeeksforGeeks. <https://www.geeksforgeeks.org/binary-search/>

Improve, K. F. (2014a, enero 7). Quick sort. GeeksforGeeks. <https://www.geeksforgeeks.org/quick-sort-algorithm/>

